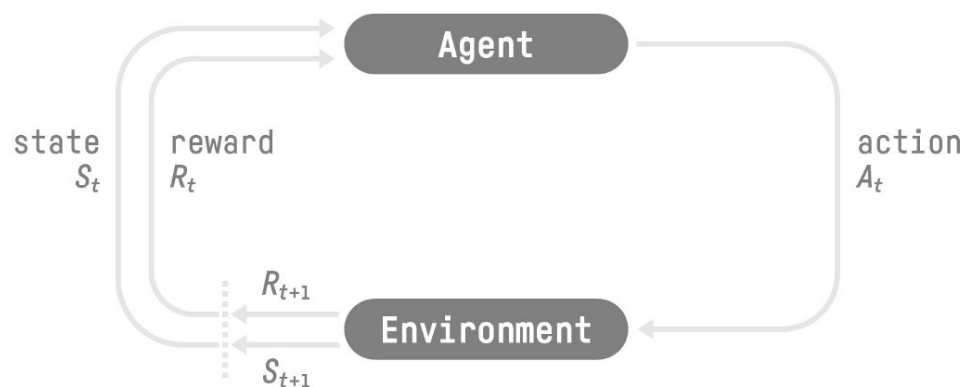


Reinforcement Learning

Naf Guo
2026

What is Reinforcement Learning



我们的目标是构建一个代理agent，能够与环境交互并采取智能决策。

其中，环境按不同的时间 t 处于不同的状态 S_t 。我们的代理agent根据所处的环境状态采取行动 A_t 。采取行动会带来两个效果，首先环境的状态会因为和agent的交互发生改变为 S_{t+1} ；其次agent会获得奖励reward R_{t+1} 。我们的目标是最大化奖励的累积，称为总回报return。

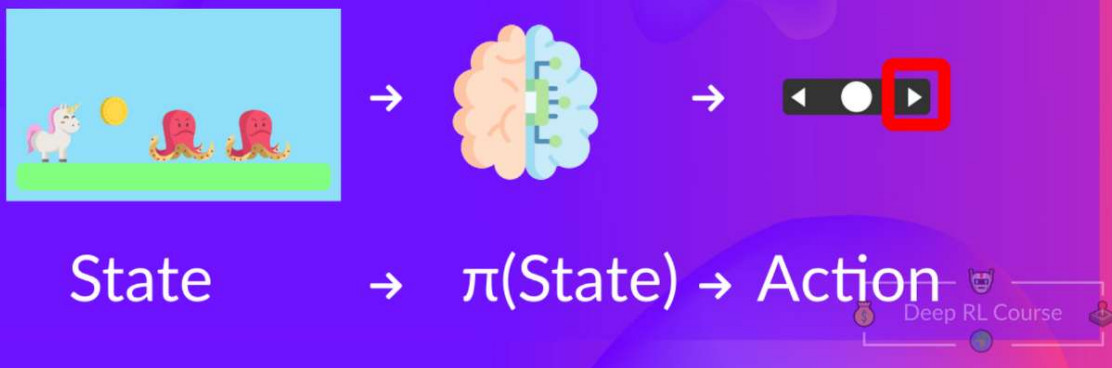
return 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

策略 Policy

The Policy π : the agent's brain

Policy π : is the brain of our Agent, it's the function that tell us what action to take given the state we are.

→ So it defines the agent behavior at a given time.



策略控制着代理的行为。

也就是我们的agent在环境处于状态S时，要采取什么行为。

强化学习的目标是要找到一种最优策略，使我们获得的期望回报最大。

也就是说，策略是一种关系，给定环境的状态，返回代理接下来采取的行动，或者行动分布

$$a_{t+1} = \pi(s_t) \quad \text{or} \quad a_{t+1} \sim \pi(\cdot | s_t)$$

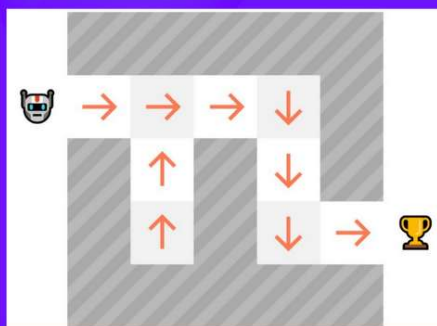
我们希望找到最优策略 π^* ，使得能获得的累积奖励也就是总回报return最大

$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

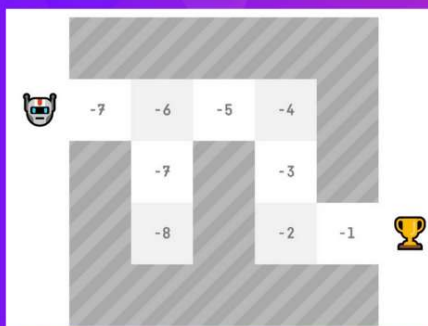
如何找到最优策略

Two approaches to find optimal policy π^* :

Policy-Based methods: train the agent to learn which action to take, given a state.



Value-Based methods: train the agent to learn which state is more valuable and take the action that leads to it.



考察一个游戏，从面具处出发，每步能移动一个格子，每走一步会有-1的reward，从而强迫代理采取尽可能少的行动；

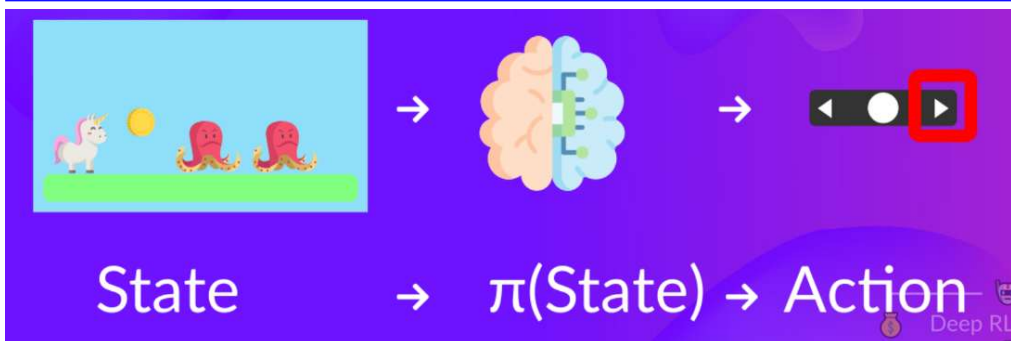
而到达奖杯处则获得游戏的胜利。

游戏的目标就是达到奖杯处。

怎么找最优的策略？

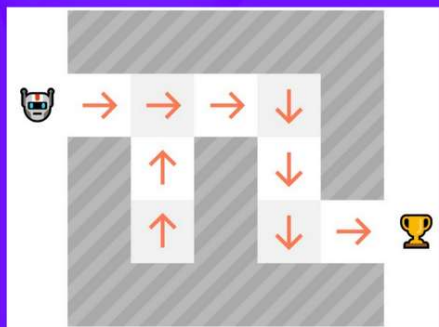
有两种方法，基于策略的方法和基于价值的方法

基于策略的方法



我们注意到策略就是一个函数，对每个可能的状态，都会给出在该状态下对应的动作或者动作分布

Policy-Based methods: train the agent to learn which **action to take**, given a state.



$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

而强化学习的目标就是找到一种策略，使得在这种策略下，代理能完成游戏并得到最大的回报

那么，我们可以训练代理，让它在每个状态，对应左图的每一个格子，都会采取一种行动，就是朝某个方向走一步，所有状态都安排了一种行动以后，我们就得到了一种策略

如果我们改变某些状态对应的行动，就得到了新策略，如果代理在新策略下能得到更多回报，那新策略就是更优的

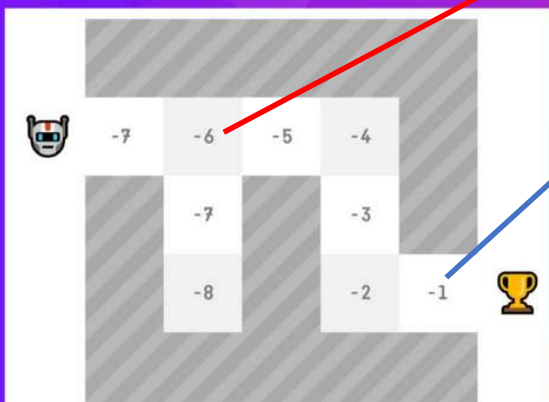
如果我们无法找到带来更大回报的策略，那我们就找到了最优策略

这是基于策略的方法

基于价值的方法

还有一种方法，就是对每一个状态，我们都为它分配一个价值分数，可以说代表这个状态的重要性

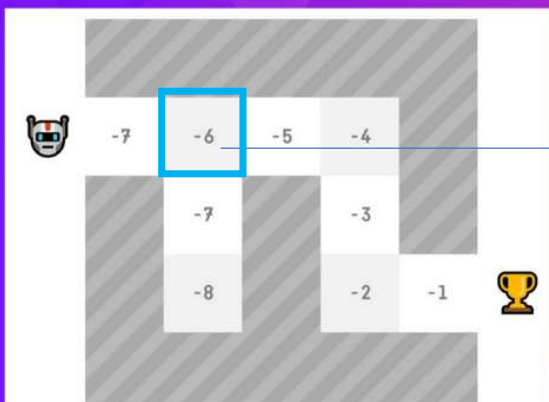
Value-Based methods: train the agent to learn which state is more valuable and take the action that leads to it.



如果到达一个状态会使你离奖励/游戏胜利更近，那这个状态应该比那些距离胜利远的状态更重要，可以被分配一个更大的分数

基于价值的方法

Value-Based methods: train the agent to learn which state is more valuable and take the action that leads to it.



还有一种方法，就是对每一个状态，我们都为它分配一个价值分数，可以说代表这个状态的重要性

如果到达一个状态会使你离奖励/游戏胜利更近，那这个状态应该比那些距离胜利远的状态更重要，可以被分配一个更大的分数

如何根据这个价值分数指导代理的行动呢？

以这个目前填写了价值分数-6的格子为例，周围有3个格子，对应的分数分别是-7，-7，-5

根据我们最开始定义的规则，每行动一步会获得-1的reward

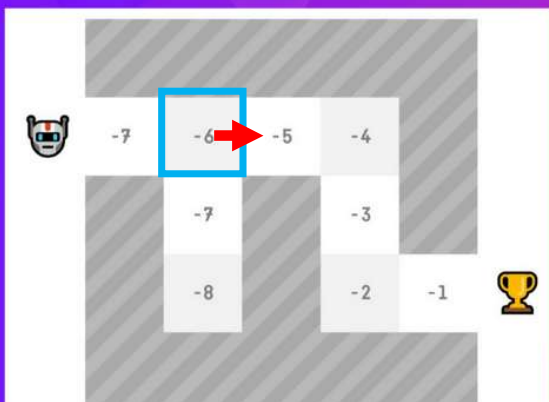
假设我们：

向右走到达价值为-5的格子，价值与移动奖励的和为-6

向下或向左走到达-7的格子，价值与移动奖励的和为-8

基于价值的方法

Value-Based methods: train the agent to learn which state is more valuable and take the action that leads to it.



还有一种方法，就是对每一个状态，我们都为它分配一个价值分数，可以说代表这个状态的重要性

如果到达一个状态会使你离奖励/游戏胜利更近，那这个状态应该比那些距离胜利远的状态更重要，可以被分配一个更大的分数

如何根据这个价值分数指导代理的行动呢？

以这个目前填写了价值分数-6的格子为例，周围有3个格子，对应的分数分别是-7，-7，-5

根据我们最开始定义的规则，每行动一步会获得-1的reward

假设我们：

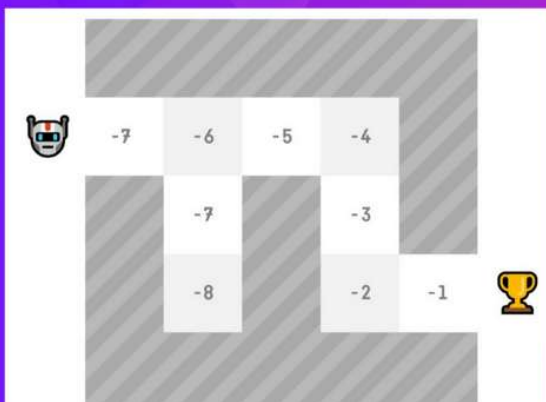
向右走到达价值为-5的格子，价值与移动奖励的和为-6

向下或向左走到达-7的格子，价值与移动奖励的和为-8

由于 $-6 > -8$ ，所以我们选择向右走

基于价值的方法

Value-Based methods: train the agent to learn which state is more valuable and take the action that leads to it.



那么，每个格子的价值如何分配才最合理呢？

我们知道，强化学习的目标是找到一套能获得最大回报的策略。

假设有两个状态，分别从这两个状态出发，一直行动到游戏结束，那它们分别能得到对应的回报

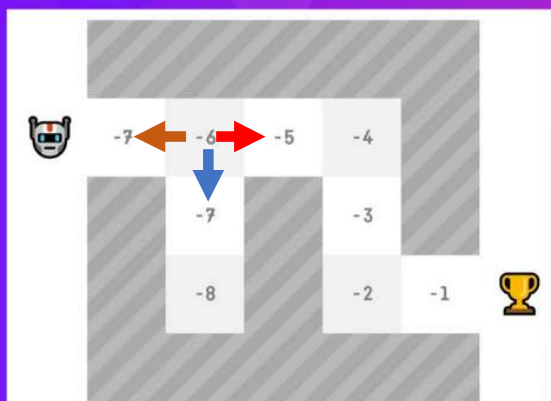
如果状态A比状态B平均得到的回报更多，那是否就代表状态A的价值更大？

因此，价值函数的一种合理的赋值方式就是分配从它出发完成游戏，能得到的回报的期望。

$$\underbrace{v_{\pi}(s)}_{\text{Value function}} = \underbrace{\mathbb{E}_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots]}_{\text{Expected discounted return}} \mid \underbrace{S_t = s}_{\text{Starting at state } s}$$

贪婪策略

Value-Based methods: train the agent to learn which state is more valuable and take the action that leads to it.



行动步reward + 到达状态价值(从该状态到游戏结束获得期望回报)



$$-1 + -5 = -6$$

总是采取使总回报最大的行动



$$-1 + -7 = -8$$

这种行为方式称为贪婪策略



$$-1 + -7 = -8$$

在一套价值函数指导下的贪婪策略如果能使得代理可以获得最大回报，那我们就得到了最优价值函数，基于最优价值函数的贪婪行动就能得到最优策略

ϵ -greedy 策略



还有一套区别于贪婪策略的策略，称为 ϵ -greedy 策略

在 ϵ 的概率下，会随机采取一个行动
在 $1-\epsilon$ 概率下，会采取贪婪行动

这种策略可能探索那些贪婪策略探索不到的状态，特别是当我们没有一套最优的价值函数时

基于价值的方法

Value-Based Methods

The value of a state is the expected discounted return the agent can get if it starts in that state, and then act according to our policy.

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

某状态对应的价值函数是从该状态出发，采取策略 π 走完剩余过程，能获得的回报的期望。

（这里用期望是因为，策略可能带有随机性，即处于某个状态，采取行动1234的概率分别是abcd云云）

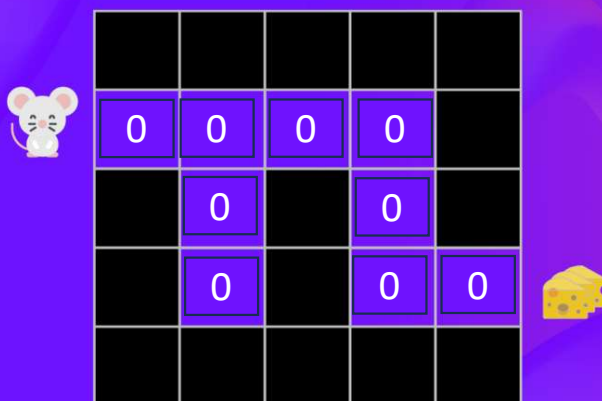
一定要注意，价值分配是和采取的策略相关的

- 价值函数 $V_{\pi}(s)$ 是在策略 π 下定义的，它衡量的是“按这个策略走，这个状态好不好”。
- 当我们有了价值函数后，就可以用贪婪策略，即每次都采取价值最大的行动来生成一个完整的策略 π

状态价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value
function

Expected discounted return

Starting
at state s

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

我们回到刚才的例子，看看这些值函数怎么算出来。

先把所有状态的价值初始化为0

为了使得老鼠采取的行动数目尽可能小，每走一步，都在reward里加入一个-1的惩罚；作为无良的人类，我放的奶酪其实是假的，因此到达奶酪的奖励为0，但一旦到达，整个过程就结束了。

设置 $\gamma = 1$ ，也就是不折扣。

小老鼠采取贪心策略，也就是只会选择价值最大的动作。

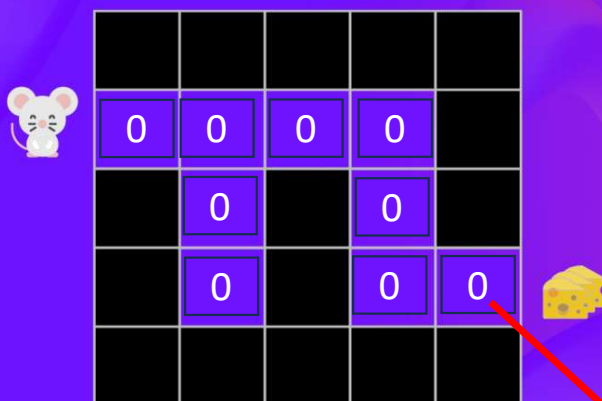
我们的目标是对每一个格子都找到到达奶酪处的最短走法，并根据最短走法确定价值函数

由于这个游戏比较简单，我们可以对每个格子穷举可能到达终点的路径，选择最短的那一条计算价值函数

状态价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

我们可以手动检查一下所有可能性，给每个位置安排上价值分数。

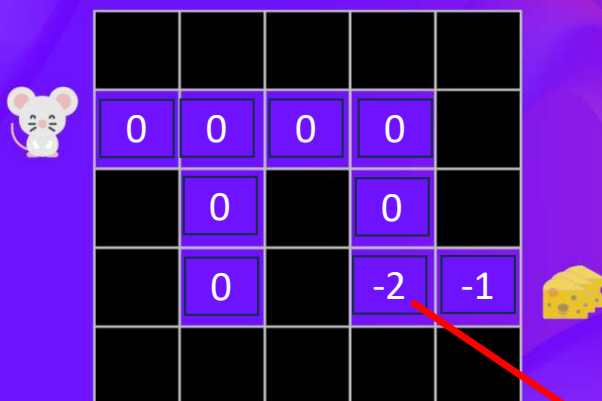
小老鼠从这个格子出发，如果向右走，则过程结束，return是-1+0=-1;

如果向左走，则每走一步有-1的罚分，但走了以后这个过程没结束，想结束必须走回头路，那又是-1的罚分，直到走到奶酪才能结束，则向左走的return是肯定比向右走的return小的。我们采取贪婪策略，所以向右走，return为-1，那这个格子的价值是-1

状态价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

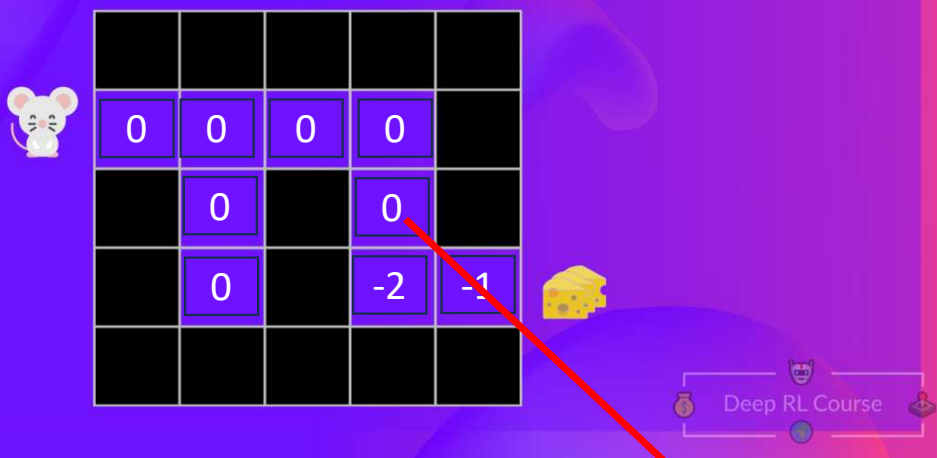
同样地，去看这个格子，它能获取最大return的方法是往右走2步到达奶酪，return是 $-1 + -1 = -2$ ，所以这个格子的价值是-2

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

状态价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

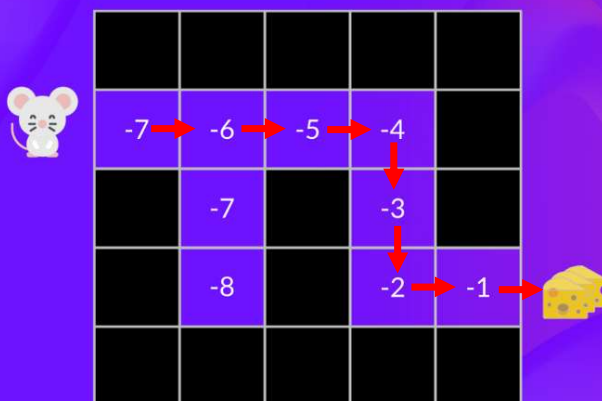
这个格子，它能获取最大return的方法是向下走1步，往右走2步到达奶酪，return是-3，所以这个格子的价值是-3

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

状态价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

总之在贪婪策略下，我们把每个格子的价值都算出来了

之后最优策略就是顺着箭头走

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

状态价值函数 The state-value function

Two types of Value-Based Methods

State Value Function:

$$V_{\pi}(s) = \mathbf{E}_{\pi}[G_t | S_t = s]$$

Value of state s

Expected return

If the agent starts at state s

And uses the policy to choose its actions for all time steps

For each state,
the state-value function outputs
the expected return
if the agent starts in that state
and then follows the policy forever after.

Deep RL Course

在上面的方法中，我们的价值函数只和状态 s 相关，称为状态价值函数，这种价值函数用 V 表示

动作价值函数 The action-value function

Two types of Value-Based Methods

Action Value Function:

$$Q_{\pi}(s, a) = \mathbf{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Value of state-action pair s, a

Expected return

If the agent starts at state s

and chooses action a

And then uses the policy to choose its actions for all time steps

For each state and action, the action-value function outputs the expected return if the agent starts in that state and takes the action and then follows the policy forever after.



还有一种价值函数称为Q函数，它是在状态 s 下，且采取动作 a ，将来能得到的回报。就是多规定了一个动作。

价值与策略

The link between Value and Policy:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Finding an optimal value function leads to having an optimal policy.

在强化学习中，无论用哪种方法，最终都会得到一个策略。基于策略的方法是直接对策略本身进行训练，从而找到最优策略 π^*

而基于价值的方法则不直接训练策略，而是先学习出最优价值，再通过“在每个状态下选择能让动作价值最大的行为”这一规则（如贪婪策略）推导出最优策略。